大模型技术之微调实战

第 1 章 LoRA 微调

1.1 环境准备

1.1.1 硬件需求

本案例的 LoRA 微调使用 RTX3090 即可完成，也可租 GPU 服务器：

- autodl：https://www.autodl.com/
- 趣算云：https://www.funhpc.com/
- 智星云：https://www.ai-galaxy.cn/
- 恒源云：https://gpushare.com/auth/reset
- ……

本课程以 AutoDL 为例，租完服务器，初次进行 conda 初始化

```
conda init
source ~/.bashrc
```

1.1.2 下载模型

创建 modelscope 环境并安装 modelscope

```
conda create -n modelscope python=3.11
conda activate modelscope
pip install modelscope filelock -i https://pypi.tuna.tsinghua.edu.cn/simple
```

model_download.py

```
from modelscope.hub.snapshot_download import snapshot_download

model_dir = snapshot_download('unsloth/Qwen3-8B-unsloth-bnb-4bit',cache_dir='/root/autodl-tmp/pretrained')
```

执行下载

```
python model_download.py
```

1.1.3 安装依赖

创建训练环境：

```
conda create -n sft311 python=3.11
conda activate sft311
```

安装所需依赖：

```
pip install unsloth transformers trl datasets -i https://pypi.tuna.tsinghua.edu.cn/simple
```

unsloth 会自动安装最新版本的 pytorch cuda 版

1.2 使用 Unsloth

1.2.1 参数说明

1）FastLanguageModel.from_pretrained

参数名	类型	必填	说明	推荐值
model_name	str	是	预训练模型名称或本地路径。 • 使用官方 4-bit 量化模型时需包含 unsloth/前缀。 • 本地路径需指向已下载的模型目录。	"unsloth/Qwen3-8B-unsloth-bnb-4bit" （根据模型尺寸调整名称）
load_in_4bit	bool	否	启用 4-bit 量化加载，**显存降低 70%**。 • Qwen3 官方已提供预量化版本，优先启用。	True（默认） （显存 ≤24GB 必选）
bnb_4bit_quant_type	str	否	量化算法类型。 • 需搭配 load_in_4bit=True 生效。	"nf4"（精度与效率最佳平衡）
bnb_4bit_compute_dtype	torch.dtype	否	量化后计算使用的数据类型。 • 根据 GPU 架构自动选择最优类型。	torch.bfloat16（A100/H100） torch.float16（T4/V100）
max_seq_length	int	否	模型支持的最大输入长度。 • Qwen3 原生支持 128K 上下文，但需显存充足。 • 低显存场景建议设为 2048 或 4096。	2048（通用场景） 8192（长文本场景）
dtype	torch.dtype	否	模型权重加载的数据类型。 • 设为 None 可自动根据硬件选择。	None（自动选择）
device_map	Str or dict	否	指定模型加载的设备分布。 • 多卡环境用 "auto"自动并行；单卡用 "cuda:0"。	"auto"（多卡） "cuda:0"（单卡）
token	str	否	Hugging Face Hub 访问令牌，用于加载需认证的模型（如 gated 模型）。	"hf_YourToken"（按需提供）
full_finetuning	bool	否	是否启用全参数微调。 • **Qwen3 推荐设为 False**（即使用 LoRA 轻量化微调）。	False（默认）

2）FastLanguageModel.get_peft_model

参数名	类型	作用	推荐值（Qwen3）	特殊说明
r	int	**LoRA 秩**：控制低秩矩阵的维度，决定可训练参数量。 ↑秩 = ↑表达能力但↑显存占用；↓秩 =↓过拟合风险但可能欠拟合。	简单任务：8 复杂任务：32-64	数据量少时建议 r=8；长文本生成任务可提升至 r=64 。
target_modules	List[str]	**适配层选择**：指定添加 LoRA 适配器的模型层。影响模型对任务特征的捕捉能力。	["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]	Qwen3 需包含 FFN 层（gate/up/down_proj）以适配门控结构 。
lora_alpha	int	**缩放因子**：控制 LoRA 输出对原始权重的调整幅度。 ↑α = ↑学习速度但需配合↓学习率。	r 或者 2*r（如 r=32→alpha=64）	避免过高（如 alpha > 128），否则易导致训练不稳定 。
lora_dropout	float	**Dropout 率**：防止过拟合的正则化手段。 0 表示禁用，>0 时在适	小数据集：0.1-0.3 大数据集：0	Unsloth 优化下可设 0，依赖早停等防过拟合 。

		配器前向传播中随机丢弃神经元。		
bias	str	**偏置项训练**：控制是否微调模型偏置参数。 选项："none"、"all"、"lora_only"。	"none"	训练偏置对效果提升有限，且增加显存占用，通常关闭 。
use_gradient_checkpointing	str/bool	**梯度检查点**：用计算时间换显存优化，支持更大批次训练。	"unsloth"（优先）或 True	Unsloth 专用优化比常规检查点节省 30% 显存 。
random_state	int	**随机种子**：固定训练随机性，确保实验可复现。	任意整数（如 3407）	生产环境必设，调试时需测试不同种子 。

3）SFTTrainer

参数名	类型	作用	推荐值（Qwen3）	特殊说明
model	PreTrainedModel	**待微调的模型**，需已添加 LoRA 适配器	必填	需 FastLanguageModel.get_peft_model 预加载 LoRA
tokenizer	PreTrainedTokenizer	**分词器**，需与模型匹配	必填	从 FastLanguageModel.from_pretrained 获取
train_dataset	Dataset	**训练数据集**，需已格式化	必填	需包含 text 字段（见 dataset_text_field）
dataset_text_field	str	**数据集中文本字段名**，存储格式化后的训练样本	"text"	必须与数据集的键名一致
max_seq_length	int	**输入序列最大长度**，与模型初始化时一致	2048（默认）或 8192	不要超过原生长度
data_collator	DataCollator	**数据整理器**，处理批次数据	DataCollatorForSeq2Seq(tokenizer)	序列生成任务必选
dataset_num_proc	int	**数据集预处理并行进程数**	2（平衡效率与内存）	Windows 需设为 1 防崩溃
packing	bool	**是否启用样本打包**，将短文本拼接成长序列	False（默认）	可加速训练但可能降低精度

4）SFTConfig

参数名	类型	作用	推荐值（Qwen3）	注意事项
per_device_train_batch_size	int	单设备批次大小（影响显存占用）	2（24GB 显存）或 4（40GB+ 显存）	需结合梯度累积调整有效批次大小
gradient_accumulation_steps	int	梯度累积步数（模拟更大批次训练）	4（默认）	有效批次大小 = batch_size × accumulation_steps

warmup_steps	int	学习率预热步数（避免初期震荡）	10（小数据集）或 50（大数据集）	占总步数 5%-10%
max_steps	int	最大训练步数（替代 num_train_epochs）	100-500（根据数据量调整）	小数据集可设 60-100
learning_rate	float	初始学习率（LoRA 微调需较低学习率）	2e-4（默认）或 5e-5（敏感任务）	过高易导致训练发散
fp16/bf16	bool	混合精度训练（加速计算并减少显存）	fp16=True（T4）或 bf16=True（A100）	通过 is_bfloat16_supported() 自动选择
optim	str	优化器类型（LoRA 推荐 8-bit 优化器）	"adamw_8bit"	显存优化显著
weight_decay	float	**L2 正则化强度**：防止过拟合，惩罚大权重值	0.01（默认）或 1e-4（敏感任务）	- 过小（<1e-5）可能无效；过大（>0.1）易欠拟合
lr_scheduler_type	str	**学习率调度策略**：动态调整学习率衰减方式	"cosine"（优先）或 "linear"	- cosine：平滑衰减，适合长序列训练 - linear：简单通用
logging_steps	int	日志记录间隔步数	10（默认）	监控损失下降趋势
output_dir	str	模型输出路径	"./outputs"	保存检查点和最终模型
report_to	str/List	日志上报平台（如 WandB/TensorBoard）	"none"（默认）或 "wandb"	生产环境建议启用监控

5）generate

参数名	作用	建议值	说明
max_new_tokens	控制生成内容的最大长度（单位：token）	1024	优于旧参数 max_length，避免截断输入文本
temperature	调节随机性：值越高输出越多样（默认 1.0）	0.3~0.7（平衡）	- 0.3：确定性高（适合问答） - 0.7：创造性高（适合创作）
top_p	核采样：仅累积概率达阈值的 token 参与采样（默认 1.0）	0.9	与 temperature 二选一调节，避免同时修改
repetition_penalty	惩罚重复 token（>1 抑制重复）	1.2	值过高可能导致语法错误
do_sample	是否启用采样（默认 False，即贪心解码）	True	需启用以配合 temperature 或 top_p 生效

1.2.2 训练代码

```
from unsloth import FastLanguageModel
import torch
from datasets import load_dataset
from trl import SFTTrainer, SFTConfig


# ================= 1、加载模型与分词器（自动优化）======================
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "/root/autodl-tmp/pretrained/unsloth/Qwen3-8B-unsloth-bnb-4bit",   # 支持模型 ID 或本地路径
```

```
    max_seq_length = 2048,              # 支持 32K+长上下文
    device_map = "auto",
    dtype = None,                       # 自动选择最优精度
    load_in_4bit = True,                # 4bit 量化节省 70%显存
    load_in_8bit = False,
    full_finetuning = False
)

# =============== 添加 Lora Adapter ================
model = FastLanguageModel.get_peft_model(
    model,
    r = 32,           # Choose any number > 0! Suggested 8, 16, 32, 64, 128
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj",],
    lora_alpha = 32,  # Best to choose alpha = rank or rank*2
    lora_dropout = 0, # Supports any, but = 0 is optimized
    bias = "none",    # Supports any, but = "none" is optimized
    # [NEW] "unsloth" uses 30% less VRAM, fits 2x larger batch sizes!
    use_gradient_checkpointing = "unsloth", # True or "unsloth" for very long context
    random_state = 3407,
    use_rslora = False,   # rank stabilized LoRA
    loftq_config = None,  # LoftQ
)

# ====================== 2、加载数据集 =========================
def convert_to_qwen_format(examples):
    conversations = []
    # 遍历每个对话样本,注意开启 batch 时，会自动套一层 list
    for conv_list in examples["conversation"]:
        # 重建符合 Qwen3 标准的消息结构
        for conv in conv_list:
            conversations.append([
                {"role": "user", "content": conv['human'].strip()},
                {"role": "assistant", "content": conv['assistant'].strip()}
            ])

    return {"conversations": conversations}

def format_func(examples):
    """应用 Qwen 对话模板"""
    formatted_texts = []

    for conv in examples["conversations"]:
        # 使用分词器的内置模板
        formatted = tokenizer.apply_chat_template(
            conv,
```

```
            tokenize=False,
            add_generation_prompt=False,
        )
        formatted_texts.append(formatted)

    return {"text": formatted_texts}

dataset = load_dataset("json", data_files="data/keywords_data_train.jsonl", split="train")
# dataset = dataset.shuffle(seed=42).select(range(1000)) #抽取 1000 条快速验证
# 格式化数据为 ChatML 格式
dataset = dataset.map(
    convert_to_qwen_format,
    batched=True,
    remove_columns=dataset.column_names
)

# 格式化成 qwen 模板
formatted_dataset = dataset.map(
    format_func,
    batched=True,
    remove_columns=["conversations"]
)

# ==================== 3.使用 trl 库的训练器 ====================
trainer = SFTTrainer(
    model = model,
    # processing_class = tokenizer,
    tokenizer = tokenizer,
    train_dataset = formatted_dataset,
    eval_dataset = None, # Can set up evaluation!
    args = SFTConfig(
        dataset_text_field = "text",
        per_device_train_batch_size = 8,
        gradient_accumulation_steps = 4, # Use GA to mimic batch size!
        warmup_steps = 5,
        num_train_epochs = 1, # Set this for 1 full training run.
        # max_steps = 30,
        learning_rate = 2e-4, # Reduce to 2e-5 for long training runs
        logging_steps = 1,
        optim = "adamw_8bit",
        weight_decay = 0.01,
        lr_scheduler_type = "linear",
        seed = 3407,
        report_to = "none", # Use this for WandB etc
    ),
)
```

```
# 显示当前内存统计信息
gpu_stats = torch.cuda.get_device_properties(0)
start_gpu_memory = round(torch.cuda.max_memory_reserved() / 1024 / 1024 / 1024, 3)
max_memory = round(gpu_stats.total_memory / 1024 / 1024 / 1024, 3)
print(f"GPU = {gpu_stats.name}. Max memory = {max_memory} GB.")
print(f"{start_gpu_memory} GB of memory reserved.")

# 开始训练
trainer_stats = trainer.train()

# 显示最终内存和时间统计信息
used_memory = round(torch.cuda.max_memory_reserved() / 1024 / 1024 / 1024, 3)
used_memory_for_lora = round(used_memory - start_gpu_memory, 3)
used_percentage = round(used_memory / max_memory * 100, 3)
lora_percentage = round(used_memory_for_lora / max_memory * 100, 3)
print(f"{trainer_stats.metrics['train_runtime']} seconds used for training.")
print(
    f"{round(trainer_stats.metrics['train_runtime']/60, 2)} minutes used for training."
)
print(f"Peak reserved memory = {used_memory} GB.")
print(f"Peak reserved memory for training = {used_memory_for_lora} GB.")
print(f"Peak reserved memory % of max memory = {used_percentage} %.")
print(f"Peak reserved memory for training % of max memory = {lora_percentage} %.")

# ==================== 4.保存微调后的模型 =====================
model.save_pretrained("/root/autodl-tmp/outputs/Qwen3-8B-sft-lora-adapter-unsloth")
tokenizer.save_pretrained("/root/autodl-tmp/outputs/Qwen3-8B-sft-lora-adapter-unsloth")


# 合并且完整保存为 16bit：几乎无精度损失，适合生产环境
# model.save_pretrained_merged("/root/autodl-tmp/outputs/Qwen3-8B-sft-fp16", tokenizer,
save_method = "merged_16bit",)

# 合并且完整保存为 4bit
# model.save_pretrained_merged("/root/autodl-tmp/outputs/Qwen3-8B-sft-int4", tokenizer,
save_method = "merged_4bit",)
```

1.2.3 推理代码

```
import torch
from unsloth import FastLanguageModel
from transformers import TextStreamer
import json

# ====================== 1. 加载基础模型与 LoRA 适配器 =====================
# 注意：必须与训练时使用相同的基础模型和配置
```

```
base_model_path = "/root/autodl-tmp/pretrained/unsloth/Qwen3-8B-unsloth-bnb-4bit"  # 基础模型路径
lora_adapter_path = "/root/autodl-tmp/outputs/Qwen3-8B-sft-lora-adapter-unsloth"  # LoRA 适配器保存路径
max_seq_length = 2048  # 与训练一致

# 加载 4bit 量化基础模型（与训练配置一致）
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=base_model_path,
    max_seq_length=max_seq_length,
    dtype=None,
    load_in_4bit=True,  # 4bit 量化
    device_map="auto",  # 自动分配设备
)


model.load_adapter(lora_adapter_path)  # 加载适配器权重

FastLanguageModel.for_inference(model) # Enable native 2x faster inference

model.eval()  # 切换到推理模式


messages = [
    {"role": "user", "content": "关键词识别: \n 梯度功能材料是基于一种全新的材料设计概念而开发的新型功能材料.陶瓷-金属 FGM 的主要结构特点是各梯度层由不同体积浓度的陶瓷和金属组成,材料在升温和降温过程中宏观梯度层间产生热应力,每一梯度层中细观增强相和基体的热物性失配将产生单层热应力,从而导致材料整体的破坏.采用云纹干涉法,对具有四个梯度层的 SiC/A1 梯度功能材料分别在机载、热载及两者共同作用下进行了应变测试,分别得到了这三种情况下每梯度层同一位置的纵向应变,横向应变和剪应变值."}
]


input_text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
)

inputs = tokenizer(input_text, return_tensors="pt").to(model.device)


# 调用 model.generate()  ======== 一次性输出
outputs = model.generate(
    **inputs,
    max_new_tokens=1024,
)
# 只输出回答部分
```

```
response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
print(response)

# 调用 model.generate()  ======== 流式输出
# streamer = TextStreamer(tokenizer,skip_prompt=True,skip_special_tokens=True)
# outputs = model.generate(
#     **inputs,
#     max_new_tokens=1024,
#     streamer=streamer,
# )
```

1.2.4 生成评估数据集

```
from unsloth import FastLanguageModel
import json

local_model_path = '/root/autodl-tmp/pretrained/unsloth/Qwen3-8B-unsloth-bnb-4bit'
lora_adapter_path = '/root/autodl-tmp/outputs/Qwen3-8B-sft-lora-adapter-unsloth'

# 1、加载基座模型、tokenizer
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=local_model_path,
    max_seq_length=2048,  # 支持 32K+长上下文
    device_map="auto",
    dtype=None,  # 自动选择最优精度
    load_in_4bit=True,  # 4bit 量化节省 70%显存
)

# 2、注入 lora 适配器
model.load_adapter(lora_adapter_path)
# 启用 unsloth 推理加速
FastLanguageModel.for_inference(model)
model.eval()

# 3、写出评估集
with open('../data/keywords_data_test.jsonl','r',encoding='utf-8') as read_file,\
    open('../data/keywords_data_test_result.jsonl','w+',encoding='utf-8') as write_file:
    # 遍历每一行
    for line in read_file.readlines():
        # 加载为 json
        line_json = json.loads(line.strip())
        # 取出 输入、输出
        user_content = line_json['conversation'][0]['human']
        assistant_content = line_json['conversation'][0]['assistant']
        # 封装成 sharegpt 样式 ==》只要输入
```

```
        messages = [
            {"role": "user", "content": user_content.strip()},
        ]

        format_messages = tokenizer.apply_chat_template(
            messages,
            tokenize=False,              # 训练时部分词，true 返回的是张量
            add_generation_prompt=True,  # 训练期间要关闭，如果是推理则设为 True
            enable_thinking=False,       # 禁用思考
        )
        inputs = tokenizer(format_messages, return_tensors='pt').to(model.device)
        outputs = model.generate(**inputs, max_new_tokens=1024)
        response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)

        write_file.write(json.dumps({"input":user_content,
"predict":response,"target":assistant_content}, ensure_ascii=False)+'\n')
```

1.3 使用 Transformer 库

1.3.1 训练代码

```
import torch
from datasets import load_dataset
from peft import LoraConfig
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    BitsAndBytesConfig
)
from trl import SFTTrainer, SFTConfig

# ===================== 1、加载模型与分词器=====================
LOCAL_MODEL_PATH = "/root/autodl-tmp/pretrained/unsloth/Qwen3-8B-unsloth-bnb-4bit"

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,              # 启用 4-bit 量化
    bnb_4bit_quant_type="nf4",      # 量化类型
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True  # 嵌套量化节省更多内存
)

model = AutoModelForCausalLM.from_pretrained(
    LOCAL_MODEL_PATH,
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True
```

```
)

tokenizer = AutoTokenizer.from_pretrained(
    LOCAL_MODEL_PATH,
    trust_remote_code=True
)


# ====================== 2. 配置 LoRA 适配器 =========================
peft_config = LoraConfig(
    r=32,  # LoRA 秩
    lora_alpha=32,  # 缩放因子
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"
    ],
    lora_dropout=0.05,  # Dropout 率
    bias="none",  # 偏置处理方式
    task_type="CAUSAL_LM"  # 任务类型
)

# 如果不用 trl 的 SFTTrainer，需要手动包装模型
# model = get_peft_model(model, peft_config)


# ==================== 2、加载数据集 =====================
def convert_to_qwen_format(examples):
    conversations = []
    # 遍历每个对话样本,注意开启 batch 时，会自动套一层 list
    for conv_list in examples["conversation"]:
        # 重建符合 Qwen3 标准的消息结构
        for conv in conv_list:
            conversations.append([
                {"role": "user", "content": conv['human'].strip()},
                {"role": "assistant", "content": conv['assistant'].strip()}
            ])

    return {"conversations": conversations}

def format_func(examples):
    """应用 Qwen 对话模板"""
    formatted_texts = []

    for conv in examples["conversations"]:
        # 使用分词器的内置模板
        formatted = tokenizer.apply_chat_template(
            conv,
```

```
            tokenize=False,
            add_generation_prompt=False,
        )
        formatted_texts.append(formatted)

    return {"text": formatted_texts}

dataset = load_dataset("json", data_files="data/keywords_data_train.jsonl", split="train")
dataset = dataset.shuffle(seed=42).select(range(100)) #抽取 100 条快速验证
# 格式化数据为 ChatML 格式
dataset = dataset.map(
    convert_to_qwen_format,
    batched=True,
    remove_columns=dataset.column_names
)

# 格式化成 qwen 模板
formatted_dataset = dataset.map(
    format_func,
    batched=True,
    remove_columns=["conversations"]
)


# ===================== 3.使用 trl 库的训练器 ====================
trainer = SFTTrainer(
    model = model,
    # tokenizer = tokenizer,   # ! 标准写法不显示传递，通过模型关联
    processing_class=tokenizer,# 新写法
    peft_config=peft_config,   # 必须传
    train_dataset = formatted_dataset,
    eval_dataset = None, # Can set up evaluation!
    args = SFTConfig(
        dataset_text_field = "text",
        per_device_train_batch_size = 1,
        gradient_accumulation_steps = 8, # Use GA to mimic batch size!
        warmup_steps = 5,
        num_train_epochs = 1, # Set this for 1 full training run.
        # max_steps = 30,
        learning_rate = 2e-4, # Reduce to 2e-5 for long training runs
        logging_steps = 1,
        optim = "adamw_8bit",
        weight_decay = 0.01,
        lr_scheduler_type = "linear",
        seed = 3407,
        report_to = "none", # Use this for WandB etc
    ),
```

```
)

# 显示当前内存统计信息
gpu_stats = torch.cuda.get_device_properties(0)
start_gpu_memory = round(torch.cuda.max_memory_reserved() / 1024 / 1024 / 1024, 3)
max_memory = round(gpu_stats.total_memory / 1024 / 1024 / 1024, 3)
print(f"GPU = {gpu_stats.name}. Max memory = {max_memory} GB.")
print(f"{start_gpu_memory} GB of memory reserved.")

# ========== 启动训练 ==========
print("===== 开始训练 =====")
trainer_stats = trainer.train()

# 显示最终内存和时间统计信息
used_memory = round(torch.cuda.max_memory_reserved() / 1024 / 1024 / 1024, 3)
used_memory_for_lora = round(used_memory - start_gpu_memory, 3)
used_percentage = round(used_memory / max_memory * 100, 3)
lora_percentage = round(used_memory_for_lora / max_memory * 100, 3)
print(f"{trainer_stats.metrics['train_runtime']} seconds used for training.")
print(
    f"{round(trainer_stats.metrics['train_runtime']/60, 2)} minutes used for training."
)
print(f"Peak reserved memory = {used_memory} GB.")
print(f"Peak reserved memory for training = {used_memory_for_lora} GB.")
print(f"Peak reserved memory % of max memory = {used_percentage} %.")
print(f"Peak reserved memory for training % of max memory = {lora_percentage} %.")


# ====================== 4.保存微调后的模型 ==========================
# 只保存 LoRA adapters，必须同时保存 tokenizer
trainer.model.save_pretrained("/root/autodl-tmp/outputs/Qwen3-8B-sft-adapter")
tokenizer.save_pretrained("/root/autodl-tmp/outputs/Qwen3-8B-sft-adapter")

# 完整保存模型和 tokenizer
# trainer.save_model("/root/autodl-tmp/outputs/Qwen3-8B-sft-lora")
```

1.3.2 推理代码

```
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    BitsAndBytesConfig
)
from peft import PeftModel, PeftConfig
import torch

base_model_path = "/root/autodl-tmp/pretrained/unsloth/Qwen3-8B-unsloth-bnb-4bit"
lora_adapter_path = "/root/autodl-tmp/outputs/Qwen3-8B-sft-adapter"
```

```python
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,              # 启用 4-bit 量化
    bnb_4bit_quant_type="nf4",      # 量化类型
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True  # 嵌套量化节省更多内存
)

base_model = AutoModelForCausalLM.from_pretrained(
    base_model_path,
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True
)

tokenizer = AutoTokenizer.from_pretrained(
    lora_adapter_path,
    trust_remote_code=True
)

#注入 LoRA 适配器权重
model = PeftModel.from_pretrained(base_model, lora_adapter_path)
model = model.merge_and_unload()  # 关键步骤：合并权重到基础模型
model.eval()  # 切换到推理模式

messages = [
    {"role": "user", "content": "关键词识别：\n 梯度功能材料是基于一种全新的材料设计概念而开发的新型功能材料.陶瓷-金属 FGM 的主要结构特点是各梯度层由不同体积浓度的陶瓷和金属组成,材料在升温和降温过程中宏观梯度层间产生热应力,每一梯度层中细观增强相和基体的热物性失配将产生单层热应力,从而导致材料整体的破坏.采用云纹干涉法,对具有四个梯度层的 SiC/A1 梯度功能材料分别在机载、热载及两者共同作用下进行了应变测试,分别得到了这三种情况下每梯度层同一位置的纵向应变,横向应变和剪应变值."}
]


input_text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
)

inputs = tokenizer(input_text, return_tensors="pt").to(model.device)
outputs = model.generate(**inputs, max_new_tokens=1024)

# 解码时跳过特殊标记
response = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(response)
```

第 2 章 全参数微调

2.1 环境准备

2.1.1 硬件需求

本案例的全参数微调使用 4 张 A800 进行分布式训练，使用 4 张 L20 进行也可勉强训练。由于训练过程中需要保存检查点，磁盘建议剩余空间>=10 倍模型大小，这里建议不低于 150G。

2.1.2 下载模型

创建 modelscope 环境并安装 modelscope

```
conda create -n modelscope python=3.11
conda activate modelscope
pip install modelscope filelock -i https://pypi.tuna.tsinghua.edu.cn/simple
```

model_download.py

```
from modelscope.hub.snapshot_download import snapshot_download


model_dir = snapshot_download('Qwen/Qwen3-8B',cache_dir='/root/autodl-tmp/pretrained')
```

执行下载

```
python model_download.py
```

2.1.3 安装依赖

创建训练环境：

```
conda create -n sft311_all python=3.11
conda activate sft311_all
```

服务器的 cuda<=12.4 手动安装适配版本的 pytorch

```
pip install torch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1 --index-url https://download.pytorch.org/whl/cu124
```

查询网址：https://pytorch.org/get-started/previous-versions/

安装所需依赖：

```
pip install transformers trl datasets -i https://pypi.tuna.tsinghua.edu.cn/simple

pip install deepspeed -i https://pypi.tuna.tsinghua.edu.cn/simple
```

优化内存碎片

```
export PYTORCH_CUDA_ALLOC_CONF="expandable_segments:True"
```

2.2 使用 deepspeed 库

2.2.1 创建配置文件

创建 deepspeed 配置文件，ds_config/deepspeed_stage_3_config.json

```
{
  "gradient_accumulation_steps": "auto",
  "gradient_clipping": "auto",
  "steps_per_print": 200,
  "train_batch_size": "auto",
```

```
"train_micro_batch_size_per_gpu": "auto",
"wall_clock_breakdown": false,
"optimizer": {
  "type": "Adam",
  "params": {
    "lr": "auto",
    "betas": "auto",
    "eps": "auto",
    "weight_decay": "auto"
  }
},
"fp16": {
  "enabled": "auto",
  "loss_scale": 0,
  "loss_scale_window": 1000,
  "initial_scale_power": 16,
  "hysteresis": 2,
  "min_loss_scale": 1
},
"zero_optimization": {
  "stage": 3,
  "offload_optimizer": {
    "device": "cpu",
    "pin_memory": true
  },
  "offload_param": {
    "device": "cpu",
    "pin_memory": true
  },
  "overlap_comm": true,
  "contiguous_gradients": true,
  "sub_group_size": 1e9,
  "reduce_bucket_size": "auto",
  "stage3_prefetch_bucket_size": "auto",
  "stage3_param_persistence_threshold": "auto",
  "stage3_max_live_parameters": 1e9,
  "stage3_max_reuse_distance": 1e9,
  "stage3_gather_16bit_weights_on_model_save": true
},
"scheduler": {
  "type": "WarmupLR",
  "params": {
    "warmup_min_lr": "auto",
    "warmup_max_lr": "auto",
    "warmup_num_steps": "auto"
  }
```

```
  },
  "activation_checkpointing": {
    "enabled": true,
    "partition_activations": true,
    "contiguous_memory_optimization": true
  }
}
```

2.2.2 训练代码

train_all.py

```
import torch
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
)
from trl import SFTTrainer, SFTConfig

# ====================== 1、加载模型与分词器========================
LOCAL_MODEL_PATH = "/root/autodl-tmp/pretrained/Qwen/Qwen3-8B"


model = AutoModelForCausalLM.from_pretrained(
    LOCAL_MODEL_PATH,
    quantization_config=None,
    # device_map="auto",
    trust_remote_code=True
)

tokenizer = AutoTokenizer.from_pretrained(
    LOCAL_MODEL_PATH,
    trust_remote_code=True
)

# ====================== 2、加载数据集 =========================
def convert_to_qwen_format(examples):
    conversations = []
    # 遍历每个对话样本,注意开启 batch 时，会自动套一层 list
    for conv_list in examples["conversation"]:
        # 重建符合 Qwen3 标准的消息结构
        for conv in conv_list:
            conversations.append([
                {"role": "user", "content": conv['human'].strip()},
                {"role": "assistant", "content": conv['assistant'].strip()}
            ])
```

```
    return {"conversations": conversations}

def format_func(examples):
    """应用 Qwen 对话模板"""
    formatted_texts = []

    for conv in examples["conversations"]:
        # 使用分词器的内置模板
        formatted = tokenizer.apply_chat_template(
            conv,
            tokenize=False,
            add_generation_prompt=False,
            enable_thinking=False
        )
        formatted_texts.append(formatted)

    return {"text": formatted_texts}


dataset = load_dataset("json", data_files="data/psychology_data.jsonl", split="train")
formatted_dataset = tokenizer.apply_chat_template(
    dataset.map(convert_to_qwen_format, batched = True)["conversations"],
    tokenize = False,
)
#格式化数据
dataset = dataset.map(
    convert_to_qwen_format,
    batched=True,
    remove_columns=["conversation_id", "category",'conversation','dataset']
)

# 格式化成 qwen 模板
formatted_dataset = dataset.map(
    format_func,
    batched=True,
    remove_columns=["conversations"]
)


# ===================== 3.使用 trl 库的训练器 ====================
trainer = SFTTrainer(
    model = model,
    processing_class=tokenizer,# 新写法
    train_dataset = formatted_dataset,
    eval_dataset = None, # Can set up evaluation!
    args = SFTConfig(
```

```
        dataset_text_field = "text",
        deepspeed = "ds_config/deepspeed_stage_3_config.json",  #添加 deepspeed 配置文件
        per_device_train_batch_size = 8,
        gradient_accumulation_steps = 8, # Use GA to mimic batch size!
        warmup_steps = 100,
        num_train_epochs = 1, # Set this for 1 full training run.
        # max_steps = 30,
        learning_rate = 3e-5, # Reduce to 2e-5 for long training runs
        logging_steps = 1,
        optim = "adamw_8bit",
        weight_decay = 0,
        lr_scheduler_type = "constant_with_warmup",
        save_total_limit=1, # 只保存一个检查点，节省磁盘空间
        seed = 3407,
        report_to = "none", # Use this for WandB etc
    ),
)

# 显示当前内存统计信息
gpu_stats = torch.cuda.get_device_properties(0)
start_gpu_memory = round(torch.cuda.max_memory_reserved() / 1024 / 1024 / 1024, 3)
max_memory = round(gpu_stats.total_memory / 1024 / 1024 / 1024, 3)
print(f"GPU = {gpu_stats.name}. Max memory = {max_memory} GB.")
print(f"{start_gpu_memory} GB of memory reserved.")

# ========== 启动训练 ==========
print("===== 开始训练 =====")
trainer_stats = trainer.train()

# 显示最终内存和时间统计信息
used_memory = round(torch.cuda.max_memory_reserved() / 1024 / 1024 / 1024, 3)
used_memory_for_lora = round(used_memory - start_gpu_memory, 3)
used_percentage = round(used_memory / max_memory * 100, 3)
lora_percentage = round(used_memory_for_lora / max_memory * 100, 3)
print(f"{trainer_stats.metrics['train_runtime']} seconds used for training.")
print(
    f"{round(trainer_stats.metrics['train_runtime']/60, 2)} minutes used for training."
)
print(f"Peak reserved memory = {used_memory} GB.")
print(f"Peak reserved memory for training = {used_memory_for_lora} GB.")
print(f"Peak reserved memory % of max memory = {used_percentage} %.")
print(f"Peak reserved memory for training % of max memory = {lora_percentage} %.")


# ====================== 保存微调后的模型 ==========================
# 完整保存模型和 tokenizer
trainer.save_model("/root/autodl-tmp/outputs/Qwen3-8B-sft-all")
```

2.2.3 启动分布式训练

使用 deepspeed 命令启动训练

```
deepspeed --num_gpus=4 train_all.py
```

2.2.4 推理代码

```
from transformers import AutoTokenizer, AutoModelForCausalLM

local_path="/root/autodl-tmp/models/Qwen3-8B-sft-all"

model = AutoModelForCausalLM.from_pretrained(
    local_path,
    device_map="auto",
    trust_remote_code=True
)

tokenizer = AutoTokenizer.from_pretrained(
    local_path,
    trust_remote_code=True
)


messages = [
    {"role": "user", "content": "关键词识别：\n 梯度功能材料是基于一种全新的材料设计概念而开发的新型功能材料.陶瓷-金属 FGM 的主要结构特点是各梯度层由不同体积浓度的陶瓷和金属组成,材料在升温和降温过程中宏观梯度层间产生热应力,每一梯度层中细观增强相和基体的热物性失配将产生单层热应力,从而导致材料整体的破坏.采用云纹干涉法,对具有四个梯度层的 SiC/A1 梯度功能材料分别在机载、热载及两者共同作用下进行了应变测试,分别得到了这三种情况下每梯度层同一位置的纵向应变,横向应变和剪应变值."}
]


input_text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
)

inputs = tokenizer(input_text, return_tensors="pt").to(model.device)
outputs = model.generate(**inputs, max_new_tokens=200)

# 解码时跳过特殊标记
response = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(response)
```

第 3 章 使用 LLaMA-Factory 微调

LLaMA Factory 是一个简单易用且高效的大型语言模型（Large Language Model）训练与微调平台。通过 LLaMA Factory，可以在无需编写任何代码的前提下，在本地完成上百种预训练模型的微调，框架特性包括：

- 模型种类：LLaMA、LLaVA、Mistral、Mixtral-MoE、Qwen、Yi、Gemma、Baichuan、ChatGLM、Phi 等等。
- 训练算法：（增量）预训练、（多模态）指令监督微调、奖励模型训练、PPO 训练、DPO 训练、KTO 训练、ORPO 训练等等。
- 运算精度：16 比特全参数微调、冻结微调、LoRA 微调和基于 AQLM/AWQ/GPTQ/LLM.int8/HQQ/EETQ 的 2/3/4/5/6/8 比特 QLoRA 微调。
- 优化算法：GaLore、BAdam、DoRA、LongLoRA、LLaMA Pro、Mixture-of-Depths、LoRA+、LoftQ 和 PiSSA。
- 加速算子：FlashAttention-2 和 Unsloth。
- 推理引擎：Transformers 和 vLLM。
- 实验监控：LlamaBoard、TensorBoard、Wandb、MLflow、SwanLab 等等。

目前企业中也是广泛使用。

Github 地址：https://github.com/hiyouga/LLaMA-Factory/blob/main/README_zh.md

官方网站：https://llamafactory.cn/

3.1 安装 LLaMA-Factory

选用之前的服务器进行即可。

3.1.1 下载源码

使用 git 下载

```
git clone https://github.com/hiyouga/LLaMA-Factory.git
```

autodl 内置代理加速：https://www.autodl.com/docs/network_turbo/

```
开启：source /etc/network_turbo
取消：unset http_proxy && unset https_proxy
```

3.1.2 安装依赖

创建 conda 环境

```
conda create -n llama_factory python=3.10
conda activate llama_factory
```

安装支持的 pytorch-cuda 版本

```
pip install torch==2.6.0 torchvision==0.21.0 torchaudio==2.6.0 --index-url https://download.pytorch.org/whl/cu126
```

安装所需依赖

```
cd LLaMA-Factory
pip install -e ".[torch,metrics]" --no-build-isolation
或
pip install -r requirements.txt
```

由于之前机器安装过多个环境，可能导致系统盘爆满，可以用如下命令清理：

```
du -sh /root/miniconda3/pkgs/ && rm -rf /root/miniconda3/pkgs/*      # conda 的历史包
du -sh /root/.local/share/Trash && rm -rf /root/.local/share/Trash   # jupyterlab 的回收站
sudo rm -rf /root/.cache/pip/*                    #删除 pip 缓存
sudo rm -rf /root/.cache/torch/hub/*              #删除 torch hub 缓存
```

验证安装成功：

```
llamafactory-cli version
```

3.2 启动 LLaMA-Factory

启动 webui

```
llamafactory-cli webui
```

正式启动

```
nohup llamafactory-cli webui >llama_factory.log 2>&1 &
```

看到“Running on local URL: http://0.0.0.0:7860”即启动成功。

如果使用 autodl 服务器启动，本地电脑访问需开启 SSH 隧道，在本地电脑执行：

```
ssh -CNg -L 7860:127.0.0.1:7860 root@connect.nmb2.seetacloud.com -p 16652
```

浏览器打开 http://localhost:7860/

3.3 准备数据集

LLaMA-Factory 目前支持 Alpaca 格式和 ShareGPT 格式的数据集。我们自己整理好格式的数据需要添加到数据集信息中。

3.3.1 Alpaca 格式

Alpaca 是基于 Meta 开源的 LLaMA 模型构建的一种微调数据集格式，特别用于 instruction-tuning，即指令微调。其数据格式的特点是提供了一个明确的任务描述（instruction）、输入（input）和输出（output）三部分。

```
{
    "instruction": "Summarize the following text.",
    "input": "Artificial intelligence (AI) is a rapidly growing field...",
```

```
    "output": "AI is an evolving technology that is growing quickly in various fields..."
}
```

- instruction: 任务的指令，告诉模型需要完成什么操作。
- input: 任务所需的输入。如果任务是开放式的或者不需要明确的输入，这一字段可以为空字符串。
- output: 任务的期望输出，也就是模型在给定指令和输入情况下需要生成的内容。

特点是结构简单，易于理解。明确分离任务指令和输入内容，适合各种自然语言处理任务，如文本生成、翻译、总结等。

3.3.2 ShareGPT 格式

ShareGPT 格式来源于通过记录 ChatGPT 与用户对话的数据集，主要用于对话系统的训练。它更侧重于多轮对话数据的收集和组织，模拟用户与 AI 之间的交互。

```
{
    "conversations": [
        {
            "role": "user",
            "content": "What is the capital of France?"
        },
        {
            "role": "assistant",
            "content": "The capital of France is Paris."
        },
        {
            "role": "user",
            "content": "Can you tell me more about Paris?"
        },
        {
            "role": "assistant",
            "content": "Paris is the largest city and the capital of France. It is known for its art, culture, and history..."
        }
    ]
}
```

- conversations: 这是一个对话列表，包含每轮对话的角色和内容。
- role: 表示对话的角色，通常为“user”表示用户，“assistant”表示 AI 助手。
- content: 具体的对话内容。

特点是结构适合对话场景，模型可以学习如何通过多轮互动来处理问题。更贴近人类与 AI 交互的方式，适用于构建和微调对话模型。

3.3.3 处理数据格式

```python
from datasets import load_dataset


def convert_to_qwen_format(examples):
    conversations = []
    # 遍历每个对话样本,注意开启 batch 时，会自动套一层 list
    for conv_list in examples["conversation"]:
        # 重建符合 Qwen3 标准的消息结构
```

```
        for conv in conv_list:
            conversations.append([
                {"role": "user", "content": conv['human'].strip()},
                {"role": "assistant", "content": conv['assistant'].strip()}
            ])

    return {"conversations": conversations}


if __name__ == '__main__':

    dataset = load_dataset("json", data_files="data/keywords_data_train.jsonl",
split="train")
    # 格式化数据为 Chatgpt 格式
    dataset = dataset.map(
        convert_to_qwen_format,
        batched=True,
        remove_columns=dataset.column_names
    )
    dataset.to_json("data/keywords_data_sharegpt.json",force_ascii=False)
```

3.3.4 添加数据集

拷贝数据文件到 data 目录中

```
Cp keywords_data_sharegpt.json /root/autodl-tmp/LLaMA-Factory/data/
```

修改 dataset_info.json 文件，添加数据集信息

```
"psychology_dialogue": {
  "file_name": "keywords_data_sharegpt.json",
  "formatting": "sharegpt",
  "columns": {
    "messages": "conversations"
  },
  "tags": {
    "role_tag": "role",
    "content_tag": "content",
    "user_tag": "user",
    "assistant_tag": "assistant"
  }
},
```

在 web ui 上就可以看到数据集

选中后，可以在页面预览数据。

3.4 启动训练

在 webui 上配置需要的训练配置，点击开始即可开始。页面可以看到日志和图表。

第 4 章 模型部署与评估

4.1 vLLM 模型部署

4.1.1 简介

相比 ollama，vLLM 更加适合企业级高并发应用场景，但对应的，显存占用也会更高，底层是基于 Pytorch 构建。

vLLM 项目主页：https://docs.vllm.ai/en/latest/index.html

Github 开源地址：https://github.com/vllm-project/vllm

从各种基准测试数据来看，同等配置下，使用 vLLM 框架与 Transformer 等传统推理库相比，其吞吐量可以提高一个数量级，这归功于以下几个特性：

- 高级 GPU 优化：利用 CUDA 和 PyTorch 最大限度地提高 GPU 利用率，从而实现更快的推理速度。Ollama 其实是对 CPU-GPU 的混合应用，但 vllm 是针对纯 GPU 的优化。
- 高级内存管理：通过 PagedAttention 算法实现对 KV cache 的高效管理，减少内存浪费，从而优化大模型的运行效率。
- 批处理功能：支持连续批处理和异步处理，从而提高多个并发请求的吞吐量。
- 安全特性：内置 API 密钥支持和适当的请求验证，不像其他完全跳过身份验证的框架。
- 易用性：vLLM 与 HuggingFace 模型无缝集成，支持多种流行的大型语言模型，并兼容 OpenAI 的 API 服务器。

4.1.2 安装

环境要求：

- OS: Linux
- Python: 3.9 -- 3.12

创建并激活 vllm 环境

```
conda create -n vllm_env python=3.11
conda activate vllm_env
```

安装 vllm

```
pip install vllm -i https://pypi.tuna.tsinghua.edu.cn/simple
```

查看安装信息

```
pip show vllm
```

4.1.3 启动

OpenAI 风格 API 响应模式

启动命令：

```
vllm serve /root/autodl-tmp/models/Qwen3-8B-sft-all --served-model-name Qwen3-8B-sft-all --max-model-len 32K
```

注意：小写的 k，单位是 1000；大写的 K，单位是 1024

如果是基础模型+lora 适配器，可加上--enable-lora --lora-modules adapter_v1=<lora 目录>

更多选项：https://docs.vllm.ai/en/stable/cli/index.html#modelconfig

正式运行应后台运行：

```
nohup vllm serve /root/autodl-tmp/models/Qwen3-8B-sft-all --served-model-name Qwen3-8B-sft-all --max-model-len 32K >vllm.log 2>&1 &
```

4.1.4 调用

1）快速测试

```
curl http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
    "model": "Qwen3-8B-sft-all",
    "messages": [
        {"role": "system", "content": "你是个乐于助人的助理。"},
        {"role": "user", "content": "我的猫死了，我很难过。"}
    ]
}'
```

2）代码调用（openai 库）

安装 openai 库

```
pip install openai -i https://pypi.tuna.tsinghua.edu.cn/simple
```

测试代码

```
from openai import OpenAI

# 连接本地
client = OpenAI(
    base_url="http://localhost:8000/v1/",
    api_key="none"  # 占位符，可忽略
)

# 多轮对话
response = client.chat.completions.create(
    model="Qwen3-8B-sft-all",  # 指定模型,必须与启动 vllm 时指定的名字一致
    messages=[
        # {"role": "user", "content": "我的猫死了，我很难过"}
        {"role": "user", "content": "量子计算对 RSA 加密的威胁与应对策略,请展示你的推理过程在think 标签中"}
    ],
    extra_body={
        "chat_template_kwargs": {"enable_thinking": False}  # 关键参数
    }
)

# print(response)
print(response.choices[0].message.content)
```

3）Langchain 调用

安装 langchain 库

```
pip install langchain-core langchain-openai -i https://pypi.tuna.tsinghua.edu.cn/simple
```

测试代码

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage

llm = ChatOpenAI(
    model="Qwen3-8B-sft-all",
    base_url="http://localhost:8000/v1/",
    api_key="none",
    temperature=0)

# 构建消息（支持多角色）
messages = [
    SystemMessage(content="你是一个专业的技术助手，回答需简洁准确"),
    HumanMessage(content="LangChain 如何调用 OpenAI 风格的大模型？")
]

# 调用模型获取响应
response = llm.invoke(messages)
print(response.content)
```

4.2 EvalScope 模型评估

EvalScope 是魔搭社区倾力打造的模型评测与性能基准测试框架，为模型评估需求提供一站式解决方案。

项目官网：https://evalscope.readthedocs.io/zh-cn/latest/index.html

Github 地址：https://github.com/modelscope/evalscope

4.2.1 安装

创建并激活 evalscope 环境

```
conda create -n evalscope python=3.11
conda activate evalscope
```

安装 evalscope

```
pip install evalscope
```

按需安装

```
pip install 'evalscope[opencompass]'   # 安装 OpenCompass backend
pip install 'evalscope[vlmeval]'       # 安装 VLMEvalKit backend
pip install 'evalscope[rag]'           # 安装 RAGEval backend
pip install 'evalscope[perf]'          # 安装 模型压测模块 依赖
pip install 'evalscope[app]'           # 安装 可视化 相关依赖

# 或可以直接输入 all，安装全部模块
pip install 'evalscope[all]'
```

4.2.2 压测

安装压测模块：

```
pip install 'evalscope[perf]'
```

模型压力测试：

```
evalscope perf \
--url "http://127.0.0.1:8000/v1/chat/completions" \
--parallel 5 \
--model Qwen3-8B-sft-all \
--number 20 \
--api openai \
--dataset openqa \
--stream
```

参数说明：

- --url 指定被测试的模型服务接口地址。这里是本地（127.0.0.1）8000 端口的 OpenAI 兼容格式聊天接口（vLLM 接口）。
- --parallel 并发请求数，即同时向模型服务发送请求，用于模拟多用户同时访问的场景。数值越大，压测强度越高。
- --model 测试的模型名称（标识用），用于在压测报告中区分不同模型。
- --number 总请求数量，本次压测共向模型发送 20 个请求（配合 --parallel 5 时，会分 4 轮发送，每轮 5 个）。
- --api openai 指定接口协议类型为 OpenAI 格式。Evalscope 会按照 OpenAI 的 API 规范（如请求体格式、参数名）构造请求，确保与模型服务兼容。
- --dataset openqa 指定压测使用的数据集。openqa 是开放问答类数据集（如常见的问答对），Evalscope 会从该数据集抽取样本作为输入，向模型发送实际请求（如用问题作为用户输入，测试模型生成回答的性能）。
- --stream 启用流式响应模式。模型会以流的形式逐段返回生成结果（类似 ChatGPT 的打字效果），而非一次性返回完整结果。该参数用于测试模型在流式输出场景下的性能。

压测指标说明：

指标项	**含义**
Time taken for tests (s)	总测试耗时
Number of concurrency	并发数
Total requests	总请求数
Succeed requests / Failed requests	成功与失败数

吞吐性能：

指标项	**含义**
Output token throughput (tok/s)	每秒生成 token 数
Total token throughput (tok/s)	总吞吐（包括输入+输出）

Request throughput (req/s)	每秒处理请求数

延迟分析：

指标项	含义
Average latency (s)	单次请求平均耗时
Average time to first token (s)	首 token 延迟
Average time per output token (s)	每个生成 token 的平均耗时

Token 分布与批处理

指标项	含义
Average input tokens per request	每次输入平均 token 数
Average output tokens per request	每次输出平均 token 数
Average package latency (s)	批处理延迟
Average package per request	每次请求中 token 包含数

4.2.3 评估

采用了 EvalScope 专门为 Qwen3 准备的 modelscope/EvalScope-Qwen3-Test 数据集进行评测，会围绕模型的推理、指令跟随、代理能力和多语言支持方面能力进行测试，该数据包含 mmlu_pro、ifeval、live_code_bench、math_500、aime24 等各著名评估数据集。

数据集地址：https://modelscope.cn/datasets/modelscope/EvalScope-Qwen3-Test/summary

也支持用自定义数据集进行评估，参考：https://evalscope.readthedocs.io/zh-cn/latest/advanced_guides/custom_dataset/llm.html#qa

```
from evalscope import TaskConfig, run_task
task_cfg = TaskConfig(
    model='Qwen3-8B-sft-all', #与 vllm 启动时指定的模型名一致
    api_url='http://127.0.0.1:8000/v1/chat/completions',
    eval_type='service',
    datasets=[
        'data_collection',
    ],
    dataset_args={
        'data_collection': {
            'dataset_id': 'modelscope/EvalScope-Qwen3-Test',
            'filters': {'remove_until': '</think>'}  # 过滤掉思考的内容
        }
    },
    eval_batch_size=128,
```

```
    generation_config={
        'max_tokens': 30000,  # 最大生成 token 数，建议设置为较大值避免输出截断
        'temperature': 0.6,  # 采样温度 (qwen 报告推荐值)
        'top_p': 0.95,  # top-p 采样 (qwen 报告推荐值)
        'top_k': 20,  # top-k 采样 (qwen 报告推荐值)
        'n': 1,  # 每个请求产生的回复数量
    },
    timeout=60000,  # 超时时间
    stream=True,  # 是否使用流式输出
    limit=200,  # 设置为 200 条数据进行测试
)
run_task(task_cfg=task_cfg)
```

4.2.4 Web 查看结果

安装 app 模块

```
pip install 'evalscope[app]'
```

启动 app 模块

```
evalscope app
```

建立 SSH 隧道

将 6006 端口改成 evalscope 端口即可，windows 中执行：

```
ssh -CNg -L 7860:127.0.0.1:7860 root@connect.bjc1.seetacloud.com -p 30119
```

访问 webui：http://localhost:7860

EvalScope 看板 (v0.17.1)
喜欢EvalScope就动动手指给我们加个star吧 😍
设置
报告根路径
outputs目录在执行评估代码所在目录
/all/outputs
请选择报告
327@@Qwen3-8B-sft-all::EvalScope-Qwen3-Test
加载并查看
请选择报告并点击 加载并查看 来查看数据
可视化
单模型
多模型
选择报告
2327@@Qwen3-8B-sft-all::EvalScope-Qwen3-Test
任务配置
数据集概览
数据集详情
数据集组成
数据集组成
mmlu_pro/law
mmlu_pro/health
mmlu_pro/math
mmlu_pro/physics
mmlu_pro/biology
ifeval/default
Score
1
0.8

第 5 章 混合数据训练

5.1 训练代码

```
from unsloth import FastLanguageModel
from unsloth.chat_templates import standardize_sharegpt
import torch
import pandas as pd
from datasets import load_dataset,Dataset
from trl import SFTTrainer, SFTConfig



# ==================== 1、加载模型与分词器（自动优化）===================
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "/root/autodl-tmp/pretrained/unsloth/Qwen3-8B-unsloth-bnb-4bit",   # 支持
模型 ID 或本地路径
    max_seq_length = 2048,              # 支持 32K+长上下文
    device_map = "auto",
    dtype = torch.float16,              # 自动选择最优精度
    load_in_4bit = True,                # 4bit 量化节省 70%显存
    load_in_8bit = False,
    full_finetuning = False
)

# =============== 添加 Lora Adapter ================
model = FastLanguageModel.get_peft_model(
    model,
    r = 32,           # Choose any number > 0! Suggested 8, 16, 32, 64, 128
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj",],
    lora_alpha = 32,  # Best to choose alpha = rank or rank*2
    lora_dropout = 0, # Supports any, but = 0 is optimized
    bias = "none",    # Supports any, but = "none" is optimized
    # [NEW] "unsloth" uses 30% less VRAM, fits 2x larger batch sizes!
    use_gradient_checkpointing = "unsloth", # True or "unsloth" for very long context
    random_state = 3407,
    use_rslora = False,   # rank stabilized LoRA
    loftq_config = None,  # LoftQ
)

# ====================== 2、加载数据集 =========================
reasoning_dataset = load_dataset("unsloth/OpenMathReasoning-mini", split = "cot")
non_reasoning_dataset = load_dataset("mlabonne/FineTome-100k", split = "train")

def generate_conversation(examples):
```

```
    problems  = examples["problem"]
    solutions = examples["generated_solution"]
    conversations = []
    for problem, solution in zip(problems, solutions):
        conversations.append([
            {"role" : "user",      "content" : problem},
            {"role" : "assistant", "content" : solution},
        ])
    return { "conversations": conversations, }

reasoning_conversations = tokenizer.apply_chat_template(
    reasoning_dataset.map(generate_conversation, batched = True)["conversations"],
    tokenize = False,
)

dataset = standardize_sharegpt(non_reasoning_dataset)

non_reasoning_conversations = tokenizer.apply_chat_template(
    dataset["conversations"],
    tokenize = False,
)

chat_percentage = 0.25
non_reasoning_subset = pd.Series(non_reasoning_conversations)
non_reasoning_subset = non_reasoning_subset.sample(
    int(len(reasoning_conversations)*(chat_percentage/(1 - chat_percentage))),
    random_state = 2407,
)
print(len(reasoning_conversations))
print(len(non_reasoning_subset))
print(len(non_reasoning_subset) / (len(non_reasoning_subset) +
len(reasoning_conversations)))

data = pd.concat([
    pd.Series(reasoning_conversations),
    pd.Series(non_reasoning_subset)
])
data.name = "text"

combined_dataset = Dataset.from_pandas(pd.DataFrame(data))
combined_dataset = combined_dataset.shuffle(seed = 3407)


# ===================== 3.使用 trl 库的训练器 ====================
trainer = SFTTrainer(
    model = model,
    # processing_class = tokenizer,
```

```
    tokenizer = tokenizer,
    train_dataset =combined_dataset,
    eval_dataset = None, # Can set up evaluation!
    args = SFTConfig(
        dataset_text_field = "text",
        per_device_train_batch_size = 2,
        gradient_accumulation_steps = 4, # Use GA to mimic batch size!
        warmup_steps = 5,
        num_train_epochs = 1, # Set this for 1 full training run.
        # max_steps = 30,
        learning_rate = 2e-4, # Reduce to 2e-5 for long training runs
        logging_steps = 1,
        optim = "adamw_8bit",
        weight_decay = 0.01,
        lr_scheduler_type = "linear",
        seed = 3407,
        report_to = "none", # Use this for WandB etc
    ),
)

# 显示当前内存统计信息
gpu_stats = torch.cuda.get_device_properties(0)
start_gpu_memory = round(torch.cuda.max_memory_reserved() / 1024 / 1024 / 1024, 3)
max_memory = round(gpu_stats.total_memory / 1024 / 1024 / 1024, 3)
print(f"GPU = {gpu_stats.name}. Max memory = {max_memory} GB.")
print(f"{start_gpu_memory} GB of memory reserved.")

# 开始训练
trainer_stats = trainer.train()

# 显示最终内存和时间统计信息
used_memory = round(torch.cuda.max_memory_reserved() / 1024 / 1024 / 1024, 3)
used_memory_for_lora = round(used_memory - start_gpu_memory, 3)
used_percentage = round(used_memory / max_memory * 100, 3)
lora_percentage = round(used_memory_for_lora / max_memory * 100, 3)
print(f"{trainer_stats.metrics['train_runtime']} seconds used for training.")
print(
    f"{round(trainer_stats.metrics['train_runtime']/60, 2)} minutes used for training."
)
print(f"Peak reserved memory = {used_memory} GB.")
print(f"Peak reserved memory for training = {used_memory_for_lora} GB.")
print(f"Peak reserved memory % of max memory = {used_percentage} %.")
print(f"Peak reserved memory for training % of max memory = {lora_percentage} %.")

# ====================== 4.保存微调后的模型 ==========================
model.save_pretrained("/root/autodl-tmp/outputs/Qwen3-8B-thinking-lora-adapter-unsloth")
```

```
tokenizer.save_pretrained("/root/autodl-tmp/outputs/Qwen3-8B-thinking-lora-adapter-unsloth")
```

5.2 推理代码

```
import torch
from unsloth import FastLanguageModel
from transformers import TextStreamer
import json

# ====================== 1. 加载基础模型与 LoRA 适配器 ======================
# 注意：必须与训练时使用相同的基础模型和配置
base_model_path = "/root/autodl-tmp/pretrained/unsloth/Qwen3-8B-unsloth-bnb-4bit"  # 基础模型路径
lora_adapter_path = "/root/autodl-tmp/outputs/Qwen3-8B-thinking-lora-adapter-unsloth"  # LoRA 适配器保存路径
max_seq_length = 2048  # 与训练一致

# 加载 4bit 量化基础模型（与训练配置一致）
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=base_model_path,
    max_seq_length=max_seq_length,
    dtype=None,
    load_in_4bit=True,  # 4bit 量化
    device_map="auto",  # 自动分配设备
)


model.load_adapter(lora_adapter_path)  # 加载适配器权重

FastLanguageModel.for_inference(model) # Enable native 2x faster inference

model.eval()  # 切换到推理模式


messages = [
    {"role": "user", "content": "Solve (x + 2)^2 = 0."}
]


input_text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
)

inputs = tokenizer(input_text, return_tensors="pt").to(model.device)
```

```python
outputs = model.generate(**inputs, max_new_tokens=1024)

# 解码时跳过特殊标记
response = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(response)
```